Verdict · Recommended architecture · The three control planes · Headless coverage matrix · Bank data & Enable Banking · Regulatory coverage · The license boundary · Your options 1–8, annotated · Core changes actually needed · Questions for Jakob & counsel

TECHNICAL ASSESSMENT · SEPTEMBER 2026 · COMPANION TO THE ACCOUNTED AUDIT

Can Accounted be your Linux?

Yes — with three qualifications that are plumbing, not architecture. This assessment answers your seven questions from the code of the running instance: how far the ledger can be driven without its UI, where the extension-API boundary actually sits, and what your proprietary layer would need from core.

Verdict

> THE ONE-PARAGRAPH ANSWER
>
> Accounted works as commodity legal-accounting infrastructure under a proprietary intelligence layer. The **ledger itself is fully machine-drivable today**: company provisioning, journal entries with direct commit, bank-transaction ingestion, AR/AP, payroll runs, reports, SIE, documents and webhooks all run on API keys with no browser. What is *not* headless splits into two very different piles: **perimeter plumbing** (creating users and API keys, fiscal year 2+, a handful of settings and filings that only exist as cookie-session routes — each a thin wrapper you could add and happily open-source) and **regulatory BankID moments** (PSD2 bank consent, Skatteverket filing signatures) that no architecture can remove — your product hosts those instants, whatever the ledger is. Nothing in the domain logic assumes a browser.

Three findings from this pass materially change your memo:

- **Your option 6 is confirmed, and it is the working path today.** A self-hosted instance with its own Enable Banking agreement (`ENABLE_BANKING_APP_ID` + `ENABLE_BANKING_PRIVATE_KEY`) calls `api.enablebanking.com` directly — no Accounted involvement, and the code explicitly un-gates the `bank_sync` paywall when it detects your own credentials (`lib/entitlements/own-credentials.ts`).
- **Your option 7 exists in the code — but isn't sellable yet.** The "hybrid: self-host + buy Accounted's connectivity" model you couldn't find public evidence for is built: connector keys (`gnubok_ck_`), hosted proxies for bank/Skatteverket/org-lookup/migration, hourly grant sync with 72-hour

offline grace, per-active-company pricing noted in the docs. The comments literally call it "the Nabu Casa model". However, the instance-side wiring is deferred (their decision log, entry on PR6b) and their sovereignty doc states keys are not yet sold. This is a direct question for Jakob.

- **The extension-API license boundary is generous in capability but ambiguous in wording.** A conforming proprietary extension can subscribe to all 80 ledger events (with full journal lines), expose its own HTTP ingress, and even commit verifikat — using only the three files the exception names. But the copyright holder's own extensions import ~120 core modules beyond those files, so the "documented Extension API" means something broader in practice than in the text. That gap is exactly what to confirm with Jakob before launch.

Recommended architecture

Your pipeline sketch, mapped onto the integration points that actually exist. Boxes with a blue border are yours and proprietary; the rest is unmodified AGPL Accounted.

YOURS · PROPRIETARY

Ingestion & economic event model

Bank feeds (your own PSD2 agreement or file parsing), invoices, documents, e-commerce. Normalized into your event model.

YOURS · PROPRIETARY

AI brain · validation · learning

Categorization, risk engine, memory, agent orchestration, your end-user app. Separate process, separate repo, any license.

↓ writes: `POST /api/v1/…/transactions/ingest` (500/batch, idempotent) · `journal-entries` + `commit` (direct, no staging) · invoices, suppliers, payroll runs

↑ reads back: 28 signed webhooks · `GET /api/events` cursor polling (durable) · v1 reports · your own Postgres (self-host)

ACCOUNTED · AGPL, UNMODIFIED

Legal ledger engine

Verifikat with atomic voucher numbers, immutability + period locks enforced by Postgres triggers, BAS 2026, storno/rättelse, document archive (7-year WORM).

ACCOUNTED · AGPL, UNMODIFIED

Legal outputs

Momsdeklaration + eSKD, AGI XML, INK2/NE SRU, K2 iXBRL årsredovisning, SIE export, payment files, behandlingshistorik.

The integration rules that make it work

- **Self-host Accounted; keep the brain in a separate process.** Self-hosting gives you what the hosted API never can: your own Postgres (direct reads, Realtime), your own `service_role` key (headless user provisioning via Supabase's admin API), and full control of env and extensions. Process separation is also your cleanest AGPL posture.
- **Write through the v1 REST API, not MCP.** v1 writes commit directly (with an optional per-key SEK ceiling for unattended commits); every MCP write stages a pending operation that needs approval. For an autonomous brain, v1 is the write plane; MCP covers the ~15 things v1 lacks (chart-of-accounts writes, period unlock, dispositioner posting) plus programmatic approval of staged operations where you want a review step.
- **Never mirror the ledger off the in-process event bus.** It is fire-and-forget with no durability or replay. Use the signed webhooks (8 retries over ~87 hours) plus cursor-polling of `GET /api/events` as the belt-and-braces feed — or, self-hosted, read your own database.
- **One platform identity drives all tenants.** A key is checked per-request against the URL's company, so one key operated by one platform user can drive every company that user owns — and `POST /api/v1/companies` provisions a complete tenant (chart, settings, first fiscal year, deadlines, cash account) in one call. Your end customers never see an Accounted login.
- **Design the BankID moments into your product.** PSD2 consent and Skatteverket signatures are legal requirements on the human, not API gaps. Your UX hosts a "connect your bank" and a "sign your VAT return" instant; everything around them stays invisible. Manual-file fallbacks (eSKD, AGI XML, SRU) exist for every filing if you'd rather ship without the Skatteverket connection at first.

The three control planes

Accounted has three distinct machine interfaces. Knowing which to use for what is most of the integration design.

PLANE	AUTH	WRITE SEMANTICS	USE IT FOR
v1 REST `/api/v1/*` · 142 operations	`gnubok_sk_` Bearer keys, 30 scopes, MFA-exempt, test-mode keys force dry-run	**Direct commit** — no staging. Idempotency-Key on writes, cursor pagination, OpenAPI generated from Zod	The brain's write plane: ingest, journal entries, AR/AP, payroll, reports, webhooks, company provisioning
MCP 165 tools over JSON-RPC	Same keys, or OAuth 2.1 + PKCE (Claude/ChatGPT connectors)	**Every write stages** a pending operation with a risk tier; approval via `gnubok_approve_pending_operation` (scope-gated, SoD acknowledgment required on one-key setups)	The ~15 gaps v1 lacks; human-in-the-loop flows; agent-native access with resources, skills, widgets

PLANE	AUTH	WRITE SEMANTICS	USE IT FOR
Internal API ~300 cookie routes	Browser session + MFA + active-company preference	Direct	Nothing, headlessly — this is the wall. Everything UI-only in the matrix below lives here

Notable deliberate design: agent auto-commit was *removed* from the product (their migration log), so the only unattended-write paths are v1 direct commits under a per-key SEK ceiling, or an explicitly SoD-acknowledged approve key. The audit trail distinguishes API-key approvals from human ones in the general ledger.

Headless coverage matrix

Full = API key alone suffices today · Partial = works with caveats or MCP-only pieces · UI-only = cookie session or browser ceremony required.

DOMAIN	STATUS	DETAIL
Company (tenant) creation	Full	`POST /api/v1/companies` seeds chart, settings, fiscal year, deadlines, cash account, membership in one call — same code path as the wizard.
Journal entries	Full	Create, batch-create, commit (direct), reverse, correct. Voucher numbering and immutability enforced by the database regardless of caller.
Bank transaction ingest	Full	500/batch, external-id + content-bridge dedup, auto-categorize and auto-match, or `raw_insert_only`. ~50k transactions/min theoretical per key.
Invoices, suppliers, customers	Full	Full lifecycle incl. credit notes, PDF, send (paid email capability). Recurring schedules and ROT/RUT are MCP-only.
Payroll runs + AGI XML	Full	Create → calculate → approve → book → paid, payslip PDFs, AGI XML retrievable for upload. Only the Skatteverket *submission* needs BankID.
Reconciliation, reports, SIE, documents, webhooks	Full	12 reconciliation ops, 16 reports, SIE import/export, document upload/link, webhook CRUD, event polling, async-operation polling.
Pending-operation approval	Partial	Programmatic via MCP only (`pending_operations:approve`); the REST pending-ops routes are cookie-only. Thin v1 wrapper would close it.
Chart of accounts, period unlock, dispositioner, settings read	Partial	MCP-staged or absent from v1; company settings have a 13-field PATCH but *no GET*.
Fiscal year 2+ creation	UI-only	The first year comes with company creation; creating the next one is a cookie route. **This bites within 12 months of go-live** — the first wrapper to add.

DOMAIN	STATUS	DETAIL
User provisioning & API-key minting	UI-only	No admin API on hosted. Self-hosted this evaporates: you own `service_role` (Supabase admin API) and can insert key rows directly.
Årsredovisning build/sign/file	UI-only	No API beyond four MCP read tools. Generators are transport-free libs, so wrappers are possible.
Bank consent, Skatteverket connect & filing signature	UI-only	Regulatory BankID ceremonies (PSD2 SCA; SKV signing link even on the MCP path). Host these moments in your product.

Bank data & Enable Banking

Three ingestion paths, all headless-compatible

- **Push your own feed** — `POST /api/v1/…/transactions/ingest` with your own `external_id`s (never re-derive them; the format is a frozen stored key). This is the natural path if your ingestion layer owns the bank relationship.
- **File import** — 14 formats (camt.053, Nordea ×2, SEB, Swedbank, Handelsbanken, Länsförsäkringar, ICA, Skandia, Lunar, Northmill, Wise ×2, generic CSV) via a v1 multipart endpoint with async polling.
- **Enable Banking sync** — the extension's unattended nightly sync runs on a plain `CRON_SECRET` bearer endpoint, fully headless; only the initial consent is a browser.

The Enable Banking answer

> **A self-hosted instance with its own Enable Banking agreement talks to `api.enablebanking.com` directly — no Accounted involvement, no connector key, and the `bank_sync` paywall switches itself off.**
>
> Verified three ways: the extension's API client mints its RS256 JWT locally from your `ENABLE_BANKING_APP_ID` / `PRIVATE_KEY`; `lib/entitlements/own-credentials.ts` makes bank sync an ungated local capability when those are set ("it must never be connector-gated" per its own docblock); and the consent callback is a route on *your* instance, registered as *your* redirect URI with Enable Banking.

The alternative — Accounted's hosted bank proxy for operators without their own agreement — is architecturally complete on the hosted side (path allowlists, per-key budgets, consent handoff with signed state, an ownership ledger that stores only hashes) but the instance side deliberately doesn't call it yet, and keys aren't sold. So for launch planning: **budget for your own Enable Banking AISP agreement**; ask Jakob about connector timing as a plan B, not the plan.

Regulatory coverage on a self-host

What generates locally with zero external credentials, what needs your own agreements, and where the manual fallback sits.

OUTPUT	GENERATES LOCALLY	DIRECT SUBMISSION	FALLBACK
Momsdeklaration (VAT)	Yes — rutor from the ledger + eSKD XML (whole-krona rules per SFL)	Skatteverket extension: needs their OAuth client + per-user BankID signing	Upload eSKD at skatteverket.se — documented as always free and working self-hosted
AGI (employer declaration)	Yes — XML via v1 API	Same BankID-gated extension path	Upload XML at Mina Sidor
INK2 / NE (income tax)	Yes — INFO.SRU + BLANKETTER.SRU, fully local	None exists — manual is the only path	Skatteverket filöverföring
Årsredovisning	K2: PDF + iXBRL locally. K3: review-draft PDF (paper filing)	Bolagsverket extension (off by default): needs your own organisationscertifikat; test env is cert-free	File the generated .xhtml/PDF manually — explicitly documented
KU10, payslips, payment files	Yes — KU10, payslip PDFs, Bankgirot LB + ISO 20022 pain.001	n/a (payment files go to your bank)	—
Skattekonto sync	—	Needs the Skatteverket connection (BankID)	Skattekonto file import exists

Both external agreements (Skatteverket API client, Bolagsverket certificate) are documented as things an operator can obtain independently — the code reads its own credentials from env and never phones Accounted for them. Skatteverket's system-level auth (no per-user BankID) is scaffolded but incomplete upstream, and even then covers reads only; personal signing on filings is a legal design, not a product gap.

The license boundary

The exception is real, irrevocable, and generous — but its wording and the project's own practice diverge. This is the part to settle with Jakob and counsel, exactly as your memo says.

What the exception grants

Proprietary extensions are permitted if they interact with Accounted *solely* through three named surfaces — `lib/extensions/types.ts` ("the Extension interface, ExtensionContext, and related

types"), `lib/events/types.ts` ("the event bus"), and the extension catch-all route — don't modify core, stay separable, and don't copy substantial core code. Modifying core (libraries, migrations, API routes, UI) keeps those modifications under full AGPL — which matches your stance that ledger improvements can be open.

What that boundary actually permits (a lot)

- **Subscribe to all 80 event types** — and the payloads are rich: `journal_entry.committed` carries the full entry including every line with accounts, amounts, dimensions and FX.
- **Expose your own HTTP ingress** — extension API routes with `skipAuth` let you implement your own bearer/HMAC auth for the brain's callbacks, cookie-free. This is exactly how the MCP server itself works.
- **Write to the ledger without importing core code** — `ctx.supabase` (a blessed capability on ExtensionContext) can insert entries and call the `commit_journal_entry` RPC directly; the database triggers enforce legality regardless. `ctx.services.ingestTransactions` is an explicitly blessed transaction-write path.
- **Be called by core without core importing you** — the services mechanism is duck-typed by name (this is how Stripe provides payment links).

Where text and practice diverge

> **The copyright holder's own extensions import ~120 core modules beyond the three named files** — including the bookkeeping engine itself (4 extensions) and the event bus implementation (31 sites, required for the documented cron pattern). The in-repo authoring guide tells extension writers that importing from `@/lib` is normal, and no CI check enforces the three-file rule. Read strictly, even first-party extensions don't conform; read as "the API as practiced and documented", the boundary is far wider than the text.

Specific ambiguities worth putting to Jakob in writing: whether `lib/events/bus.ts` is inside "the event bus" grant; whether "related types" covers the full domain model in `@/types` (event payloads require it); whether registry lookups are in-bounds; and one boundary defect — registering a settings-panel *component* requires editing a core file, which a strictly conforming extension can't do.

Your two clean postures

A · Network-only (zero linking)

The brain integrates exclusively over v1 REST + webhooks + MCP. No extension, no shared process, no linking question at all — the AGPL analysis reduces to "we run an unmodified AGPL service", the least arguable position available. Cost: no in-process synchronous hooks; event flow is webhook/polling latency.

B · Thin shim extension

A minimal extension that only forwards events to the brain and exposes callback routes, written against the three named files plus `@supabase/supabase-js`. Gets you in-process hooks and the blessed write paths. Residual risk is the wording ambiguities above — settle them with Jakob first.

Either way, your nuance stands and the code confirms it: the exception governs *extensions*; "proprietary embedding requires a commercial licence" on their site refers to something the

exception doesn't cover. Get the intended boundary in writing, then have counsel bless the chosen posture. Also note AGPL §13 still applies to the Accounted instance itself: users of your service must be offered its (unmodified or modified) source — which costs you nothing under this architecture.

Your options 1–8, annotated from the code

#	YOUR OPTION	WHAT THE CODE SAYS
1	Accounted hosted / API	Viable for MVP: v1 + webhooks work identically against hosted. Limits: 100 requests/min per key (only changeable by them), 10 keys/company, no direct DB access, user/key provisioning stays manual, and their trial/billing applies per company you create.
2	White-label hosted	The machinery exists and is documented (branding env family, byrå brand system, whitelabel domains). Matches your "fastest test" read; doesn't get you the invisible-bookkeeping UX you want long-term.
3	**Self-host core + separate proprietary brain**	**Confirmed as the strongest fit** — see the recommended architecture. Self-host unlocks headless user provisioning and direct DB reads; the brain stays proprietary in a separate process; the ~10 core gaps are thin wrappers you can upstream or keep as an open fork branch.
4	Self-host and modify core	Legally fine, modifications stay AGPL — consistent with your "the moat is not a secret debit/credit function" stance. The gaps you'd close (v1 wrappers for pending-ops, fiscal years, eSKD/SRU/årsredovisning, key management) are exactly the kind of change the project would plausibly accept upstream, which is cheaper than carrying a fork.
5	Commercial license from Jakob	Not needed for architecture 3/A. Becomes relevant only if you want to embed core into a closed distribution or escape §13 source-offering. The extension exception's existence and the connector model both signal a maintainer who has priced these lines deliberately — worth the conversation regardless, for pricing at your 100/1k/10k/100k scale.
6	Self-host + own integration agreements	**Confirmed and working today** for Enable Banking (fully direct), AI provider (Bedrock/Anthropic/OpenAI-compatible/local), email (Resend/SMTP), storage (your Supabase). Skatteverket and Bolagsverket agreements are likewise operator-obtainable per the docs.
7	Hybrid: self-host + buy Accounted's connectivity	**Exists in the code** (connector keys, hosted proxies, grant sync, per-company pricing, "Nabu Casa model" in the comments) **but is not purchasable yet** — instance-side wiring deferred, keys not sold. Ask Jakob for the timeline; treat as plan B behind option 6.
8	Build your own engine	Correctly a fallback. The audit's scale numbers (730 migrations, 22 legal outputs, trigger-enforced compliance) are the cost of recreating it. Studying Accounted as a reference while writing original code is legitimate; copying implementation is not.

Core changes actually needed

The complete list for a headless SaaS on top — all plumbing, each a thin wrapper over an already transport-free library. These would be AGPL (fine, per your memo) and are strong

upstream-PR candidates.

CHANGE	EFFORT	WHY
v1 wrapper: create/adjust fiscal periods	Small	The one that bites first — year 2 cannot be opened headlessly today.
v1 wrapper: pending-operations list/commit/reject	Small	Executor is transport-free; today MCP is the only machine path to approvals.
v1 wrappers: eSKD file, INK2/NE SRU, årsredovisning build/PDF/iXBRL, bank-import undo	Small each	Generators all take `(supabase, companyId, …)` already.
Company settings GET + wider PATCH	Small	v1 can write 13 fields but read none; VAT/method/fiscal config is UI-only after creation.
API-key management path	Small / none	Self-hosted you can insert key rows directly (SHA-256 of the token) and set `rate_limit_rpm` in the DB; a v1 route is the tidy version. The 10-key cap and 100 RPM default are single literals.
Optional: UI-off switch	Tiny	No headless flag exists; today it's "don't link to it". An env-gated 404 for non-API paths in the proxy would formalize it.

Nothing on this list touches the accounting engine, migrations, or the enforcement triggers. That is the strongest signal in the whole assessment: the domain core is already headless — only the perimeter was built UI-first.

Questions for Jakob & counsel

For Jakob / Accounted

- Does "documented Extension API" mean the three named files strictly, or `@/lib` as your own extensions and authoring guide practice it? Would you confirm the intended boundary in writing (and ideally fix the settings-panel registration so a conforming extension never edits core)?
- Is a separate-process system that integrates only over v1 REST + webhooks considered outside the "proprietary embedding requires a commercial licence" language on the site?
- Connector keys: timeline for the instance-side wiring and sale? Pricing at 100 / 1,000 / 10,000 / 100,000 companies, for both connector and any commercial-license scenario?
- Would you accept upstream PRs for the headless perimeter (v1 fiscal periods, pending-ops, filings, key management)?

For counsel

- Bless one posture: network-only integration (A) or shim extension (B), with the ctx.supabase / bus.ts / related-types ambiguities resolved against whatever Jakob confirms.

- AGPL §13 compliance mechanics for your deployment: how you offer the Accounted instance's source (including any perimeter patches) to your users.
- Trademark: "Accounted"/"Gnubok" naming in your materials, given full rebranding via the white-label system.

Assessed 2026-09-03 against the repository at migration 20260831200000 (730 migrations) and the live local instance. Companion to the full Accounted Audit page. Legal statements here are engineering readings of the license text and code — confirm with counsel before commercial launch.